

mobiTrace

Plan de Continuité Informatique

PCI — Architecture, surveillance et procédures de reprise

Référence	TRACE-PCI-2026
Version	1.0
Date	08/08/2026
Auteur	Équipe technique mobiTrace / DGFIP

Résumé exécutif

Ce document constitue le Plan de Continuité Informatique (PCI) de mobiTrace, application souveraine de gestion d'inventaire mobilier de la DGFIP. Il décrit l'architecture de référence, les procédures de surveillance et de détection d'incidents, les modes dégradés, les séquences de redémarrage des services et le calendrier de maintenance préventive.

Objectifs : RTO = 4 heures (délai maximal de remise en service), RPO = 12 heures (perte de données maximale tolérée). Le PCI est complémentaire du PCA (Plan de Continuité d'Activité) et du PRA (Plan de Reprise d'Activité).

Avertissement — Ce document contient des informations relatives à l'architecture et aux procédures de sécurité de l'infrastructure. Ne pas diffuser hors des équipes habilitées.

1. Architecture de référence

1.1 Cartographie des composants

Composant	Machine	Port / Config	Dépendance	Criticité
Nginx (reverse proxy / SSL)	Serveur TRACE	80 / 443	—	Critique
PostgREST (trace-api)	Serveur TRACE	3000 (loopback)	postgresql	Critique
PostgreSQL 17	Serveur TRACE	5432 (loopback)	—	Critique
Fail2Ban	Serveur TRACE	—	nginx logs	Élevée
Montage NFS	Serveur TRACE	/mnt/savetrace	Serveur NFS	Élevée
Serveur NFS	Machine NFS	2049	—	Élevée
mobitrace-print (optionnel)	Serveur TRACE	5050	cups	Faible

1.2 Points de défaillance uniques (SPOF)

L'architecture actuelle présente les points de défaillance uniques suivants, par ordre de criticité :

SPOF	Impact	Mitigation actuelle
Serveur TRACE (machine unique)	Arrêt total de l'application	Restauration sur nouvelle machine via PRA
Base PostgreSQL (instance unique)	Perte d'accès aux données	Sauvegardes NFS 2x/jour, restore < 1h
Serveur NFS (machine unique)	Perte des sauvegardes récentes	Rétention 30j, backup local temporaire
Secret JWT (point de confiance)	Compromission de toutes les sessions	Rotation manuelle (procédure §3.5)

L'absence de haute disponibilité (HA) est un choix architectural délibéré pour minimiser la complexité. Le RTO de 4h est atteignable par restauration sur bare metal, sans clustering.

1.3 Flux de données et dépendances

Séquence d'appel pour une requête HTTP standard :

```
Navigateur → HTTPS:443 → Nginx → vérif. cookie JWT → HTTP:3000 → PostgREST
PostgREST → vérifie JWT (secret dans auth.secrets) → SQL → PostgreSQL:5432
PostgreSQL → logique métier (triggers, RBAC, RLS) → résultat → PostgREST → Nginx → Navigateur
```

En cas de panne d'un composant, toute la chaîne en aval est affectée. PostgreSQL est le composant fondamental : sans lui, aucun autre service n'est fonctionnel.

2. Surveillance et détection des incidents

2.1 Sondes actives en place

Sonde	Mécanisme	Fréquence	Alerte
Sauvegarde NFS	Script mobitrace_backup_survey.	Quotidien 08h00	Email : OK / AVERTISSEMENT / ALERTE CRITI
Redémarrage services	Restart=always dans les units sy	Continu	Journal systemd
Anti-bruteforce	Fail2Ban filtre nginx-trace	Temps réel	Journal fail2ban

2.2 Procédure de diagnostic — Arbre décisionnel

Face à un signalement d'indisponibilité, suivre cette séquence dans l'ordre :

```
# Étape 1 : vérifier les services systemd
systemctl status nginx trace-api postgresql fail2ban

# Étape 2 : tester la connectivité applicative
curl -sk https://localhost/api/rpc/me | head -c 200

# Étape 3 : vérifier les logs récents
journalctl -u trace-api -u nginx --since '30 min ago' | tail -50

# Étape 4 : vérifier PostgreSQL
sudo -u postgres psql -c 'SELECT version();'

# Étape 5 : vérifier le montage NFS
df -h /mnt/savetrace && ls -lt /mnt/savetrace | head -5
```

2.3 Indicateurs d'alerte

Indicateur	Seuil d'alerte	Action
Codes HTTP 5xx sur /api/	> 5 en 5 min	Diagnostic §2.2
Temps de réponse /api/	> 5 secondes	Vérifier pg_stat_activity
Connexions PostgreSQL	> 80 % max_conn.	Vérifier PostgREST
Espace disque /mnt/savetrace	< 20 % libre	Purge manuelle ou extension NFS
Email sonde 08h00 absent	Absence > 24h	Vérifier serveur NFS et cron
Fail2Ban bans	> 10 IPs/heure	Alerte sécurité → §3.5

3. Procédures de continuité technique

3.1 Séquence de redémarrage des services (priorité)

En cas de redémarrage nécessaire, respecter impérativement cet ordre :

Ordre	Service	Commande
1	PostgreSQL (fondation)	sudo systemctl restart postgresql
2	trace-api (PostgREST)	sudo systemctl restart trace-api
3	Nginx (point d'entrée)	sudo systemctl reload nginx
4	Fail2Ban (sécurité)	sudo systemctl restart fail2ban
5	mobitrace-print (optionnel)	sudo systemctl restart mobitrace-print

```
# Redémarrage complet de la pile (ordre correct)
sudo systemctl restart postgresql
sleep 3
sudo systemctl restart trace-api
sudo systemctl reload nginx
sudo systemctl restart fail2ban
# Vérification globale
systemctl is-active postgresql trace-api nginx fail2ban
```

3.2 Mode dégradé — PostgREST indisponible

Si trace-api ne démarre pas mais que PostgreSQL fonctionne, les données sont préservées. L'interface web est inaccessible mais aucune donnée n'est perdue.

- Diagnostic : journalctl -u trace-api -n 50
- Cause fréquente 1 : désynchronisation du secret JWT → voir §3.3
- Cause fréquente 2 : /etc/trace-api.conf corrompu → restaurer depuis sauvegarde du paquet
- Vérification config : sudo -u www-data /usr/local/bin/postgrest /etc/trace-api.conf

3.3 Synchronisation du secret JWT

Après une restauration de base de données, le secret JWT dans auth.secrets peut différer de celui dans /etc/trace-api.conf, bloquant toutes les authentifications (erreur 401 généralisée). Le script trace_restore.sh gère cette synchronisation automatiquement. En cas de désynchronisation manuelle :

```
# Lire le secret JWT depuis la base
JWT=$(sudo -u postgres psql -d parc_mobilier -t -A \
    -c "SELECT value FROM auth.secrets WHERE key = 'jwt_secret';")

# Mettre à jour la configuration PostgREST
sudo sed -i "s|^jwt-secret = .*|jwt-secret = \"$JWT\"|" /etc/trace-api.conf

# Redémarrer PostgREST
sudo systemctl restart trace-api
echo 'Secret JWT synchronisé.'
```

3.4 Remontage NFS

Si le montage NFS est perdu (timeout, redémarrage réseau) :

```
# Vérifier l'état du montage
mountpoint -q /mnt/savetrace && echo 'Monté' || echo 'Non monté'
```

```
# Forcer le remontage
sudo systemctl restart mnt-savetrace.automount
# ou
sudo mount /mnt/savetrace

# Vérifier la connectivité vers le serveur NFS
showmount -e <IP_SERVEUR_NFS>
```

3.5 Procédure en cas d'incident de sécurité

En cas de suspicion de compromission (intrusion, bruteforce réussi, comportement anormal) :

- Étape 1 — Isolation immédiate : bloquer l'accès externe (iptables ou arrêt Nginx)
- Étape 2 — Préservation des preuves : copie des logs avant toute action corrective
- Étape 3 — Rotation du secret JWT : toutes les sessions sont invalidées
- Étape 4 — Réinitialisation des mots de passe des comptes suspects
- Étape 5 — Analyse des audit_logs pour retracer l'activité anormale
- Étape 6 — Notification RSSI DGFIP

```
# Rotation du secret JWT (invalide toutes les sessions actives)
NEW_SECRET=$(tr -dc 'A-Za-z0-9_' < /dev/urandom | head -c 48)
sudo -u postgres psql -d parc_mobilier \
    -c "UPDATE auth.secrets SET value = '$NEW_SECRET' WHERE key = 'jwt_secret';"
sudo sed -i "s|^jwt-secret = .*|jwt-secret = \"$NEW_SECRET\"|" /etc/trace-api.conf
sudo systemctl restart trace-api
```

4. Maintenance préventive et tests

4.1 Calendrier de maintenance

Fréquence	Action	Responsable	Durée
Quotidien (auto)	Sauvegarde NFS 02h00 et 13h00	Cron	Auto
Quotidien (auto)	Sonde surveillance email 08h00	Cron	Auto
Mensuel	Vérification espace disque NFS	Équipe technique	15 min
Mensuel	Contrôle des bans Fail2Ban actifs	Équipe technique	15 min
Trimestriel	Test de restauration d'un dump (env. de tes	Équipe technique	2h
Semestriel	Mise à jour des paquets Debian (apt upgrad	Équipe technique	1h
Annuel	Rotation du secret JWT	Équipe technique	30 min
Annuel	Test complet PCA/PRA	Équipe tech. + métier	4h

4.2 Procédure de mise à jour des paquets

La mise à jour des paquets système doit suivre cette procédure pour éviter les régressions :

- Avant la mise à jour : effectuer une sauvegarde manuelle (sudo /usr/local/bin/trace_backup.sh)
- Vérifier la compatibilité PostgreSQL → PostgREST avant toute mise à jour majeure
- Appliquer en heures creuses (hors 08h30-18h00, hors période de saisie intensive)
- Tester l'authentification et la consultation après chaque mise à jour

4.3 Vérification de l'intégrité des sauvegardes

Un dump PostgreSQL non testé est une sauvegarde de valeur inconnue. Procédure de test trimestrielle sur environnement de test (jamais en production) :

```
# Sur une machine de test (jamais en production)
# 1. Copier le dump depuis NFS
scp serveur-nfs:/var/nfs/backupTRACE/trace_backup_YYYY-MM-DD.dump /tmp/

# 2. Créer une base de test
sudo -u postgres createdb parc_mobilier_test

# 3. Restaurer
sudo -u postgres pg_restore --clean --if-exists \
    -d parc_mobilier_test /tmp/trace_backup_YYYY-MM-DD.dump

# 4. Vérifier les tables principales
sudo -u postgres psql -d parc_mobilier_test \
    -c 'SELECT count(*) FROM mobiliers; SELECT count(*) FROM utilisateurs;'

# 5. Nettoyer
sudo -u postgres dropdb parc_mobilier_test
```

5. Fiche de référence rapide — Garde technique

Cette page est destinée à l'astreinte technique. Elle synthétise les actions immédiates en cas d'incident sans nécessiter la lecture complète du PCI.

5.1 Contacts d'urgence

Rôle	Contact	Disponibilité
Support technique mobiTrace	equipe.support@dgfip.finances.gouv.fr	Heures ouvrées
Responsable PCA	À compléter	H24 si PCA activé
RSSI DGFIP	À compléter	H24 si incident SSI
Astreinte infrastructure DGFIP	À compléter	H24

5.2 Commandes de diagnostic rapide

```
# Statut global de tous les services mobiTrace
systemctl status nginx trace-api postgresql fail2ban mobitrace-print 2>/dev/null

# Test fonctionnel (réponse attendue : 200 OK ou 401 si non connecté)
curl -sk -o /dev/null -w '%{http_code}' https://localhost/api/rpc/me

# 10 dernières erreurs applicatives
journalctl -u trace-api -u nginx -p err --since '1h ago'

# Dernières sauvegardes disponibles
ls -lth /mnt/savetrace/trace_backup_*.dump 2>/dev/null | head -5

# Connexions PostgreSQL actives
sudo -u postgres psql -c 'SELECT count(*) FROM pg_stat_activity;'
```

5.3 Décision rapide — Que faire ?

Symptôme observé	Action immédiate
Interface web inaccessible (ERR_CONNECTION_REFUSED)	sudo systemctl restart nginx
Erreur 502 Bad Gateway	sudo systemctl restart trace-api (puis nginx si nécessaire)
Erreur 401 sur toutes les requêtes	Vérifier désynchronisation JWT (§3.3)
PostgreSQL ne répond pas	sudo systemctl restart postgresql → puis trace-api
NFS non monté	sudo mount /mnt/savetrace
Fail2Ban bloque des IPs légitimes	sudo fail2ban-client set nginx-trace unbanip <IP>
Email sonde absent depuis > 24h	Vérifier serveur NFS et cron sur machine NFS
Performance très dégradée	psql -c "SELECT * FROM pg_stat_activity WHERE state='active';"

Document généré automatiquement par generate_pci_pdf.py (dépôt mobiTrace). Pour toute question : equipe.support@dgfip.finances.gouv.fr